First, make sure that you have downloaded the accompanying code for the book and installed the necessary libraries. Detailed instructions on how to do this are included in the appendix. We will be using the code included in the Jupyter notebook `CH02/01 - Gaussian processes.ipynb`.

2.4.1 Setting up the training data

Before we go straight to implementing the code for our GP model, let's first spend some time creating an objective function we'd like to model and a training data set. To do this, we need to import PyTorch for calculating and manipulating tensors, and Matplotlib for data visualization.

```
import torch
import matplotlib.pyplot as plt
```

Our objective function in this example is the one-dimensional Forrester function, which has the following formula.

$$f(x) = -\frac{1}{5}(x+1)^2 \sin(2x+2) + 1$$

This is implemented as follows:

```
def forrester_1d(x):
    y = -((x + 1) ** 2) * torch.sin(2 * x + 2) / 5 + 1
    return y.squeeze(-1)
```

Let us quickly plot this function in a graph. Here, we restrict ourselves to the domain between -3 and 3 and compute this Forrester function on a dense grid of one hundred points in this range. This plot is generated by the following code, which produces Figure 2.15:

```
xs = torch.linspace(-3, 3, 101).unsqueeze(1)
ys = forrester_1d(xs)

plt.figure(figsize=(8, 6))

plt.plot(xs, ys, label="objective", c="r")
plt.scatter(train_x, train_y, marker="x", c="k", label="observations")

plt.legend(fontsize=15);
```

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>